VULNERABILITY ASSESSMENT

AND

PENETRATION TESTING REPORT

FOR

DIVA ANDROID APPLICATION

STATIC APPLICATION SECURITY TESTING (SAST)

FROM

SANIYA SHAIKH

1. Executive Summary

This report presents the security analysis of the DIVA (Damn Insecure and Vulnerable App) Android application. The assessment was conducted using static analysis techniques to identify potential security weaknesses in the application's code, configuration, storage, and resources.

Tools such as APKTool, JADX, and command-line utilities were used to inspect the APK, review its manifest, analyze Java and Smali code, and identify potentially exposed sensitive information.

The analysis identified several security issues, including hardcoded credentials, insecure data storage, sensitive information in application logs, debugging enabled, and a hardcoded secret key. These findings highlight areas where application security could be improved.

The purpose of this report is to document the findings, explain their potential impact, and provide recommendations for improving the application's security.

2. Objective

The main objectives of this security analysis were:

- To examine the internal structure of the DIVA APK.
- To analyze the AndroidManifest.xml file and identify insecure configuration settings.
- To identify hardcoded credentials, secret keys, and other sensitive information.
- To examine how the application stores sensitive data.
- To review application code for potential security weaknesses.
- To identify potential risks and recommend suitable remediation measures.

3. Scope of Assessment

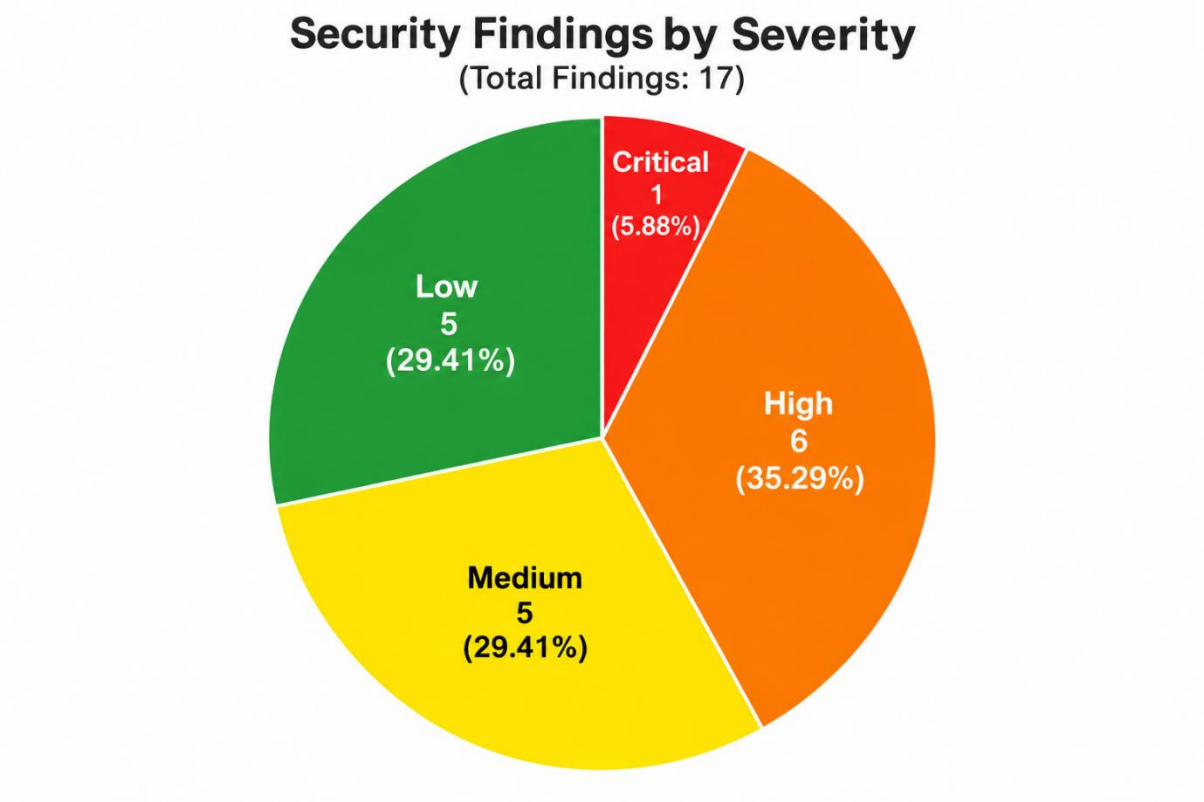
The assessment focused on static analysis of the DIVA Android application.

Scope Area	Description
APK Structure	Examined the internal files and directories of the APK.
Manifest Analysis	Reviewed application settings, debugging status, backup configuration, and permissions.
Resource Analysis	Searched resource files for potentially sensitive information.
DEX Analysis	Extracted readable strings from compiled application code.
Java Source Code	Reviewed decompiled source files for security weaknesses.
Smali Analysis	Examined bytecode for hardcoded values and access checks.
Native Libraries	Inspected native libraries for readable strings and potential secrets.
Data Storage	Reviewed SQLite database and temporary file storage practices.
Application Logs	Checked whether sensitive information was written to system logs.
APK Signing	Examined signing-related files and certificate information.

4. Table of Contents and Risk Summary

Sr. No.	Finding Name	Risk Level	OWASP Mobile Top 10
1	Hardcoded API Credentials in Application Code	Critical	M1 – Improper Credential Usage
2	Debuggable Application Enabled	High	M8 – Security Misconfiguration
3	Hardcoded Sensitive Information in DEX File	High	M1 – Improper Credential Usage
4	Insecure SQLite Database Storage	High	M9 – Insecure Data Storage
5	Insecure Temporary File Storage	High	M9 – Insecure Data Storage
6	Sensitive Information in Application Logs	High	M9 – Insecure Data Storage
7	Hardcoded Secret Key in Smali Code	High	M1 – Improper Credential Usage
8	Outdated Android Target SDK Version	Medium	M8 – Security Misconfiguration
9	Broad External Storage Permissions	Medium	M9 – Insecure Data Storage
10	Insecure Application Configuration in AndroidManifest.xml	Medium	M8 – Security Misconfiguration; potential M9
11	Potential Hardcoded Secret in Native Library	Medium	M1 – Improper Credential Usage (potential)
12	APK Signed with a Debug Certificate	Medium	M7 – Insufficient Binary Protections (potential)
13	Application Information Disclosure	Low	No direct category
14	APK Internal Structure and Component Identification	Low	No direct category
15	APK Signing Files Identified	Low	M7 – Insufficient Binary Protections (potential)
16	Successful APK Decompilation and Source Code Identification	Low	No direct category
17	Sensitive Information Indicators in Application Resources	Low	M9 – Insecure Data Storage (potential)

5. Graphical summary



6. Technical Findings

Step 1: Create the lab directory

mkdir -p ~/Mobile-Pentesting/APKs

Step 2: Navigate to the APK directory

cd ~/Mobile-Pentesting/APKs

Step 3: Clone the DIVA APK repository

git clone https://github.com/0xArab/diva-apk-file.git

Step 4: Enter the downloaded repository

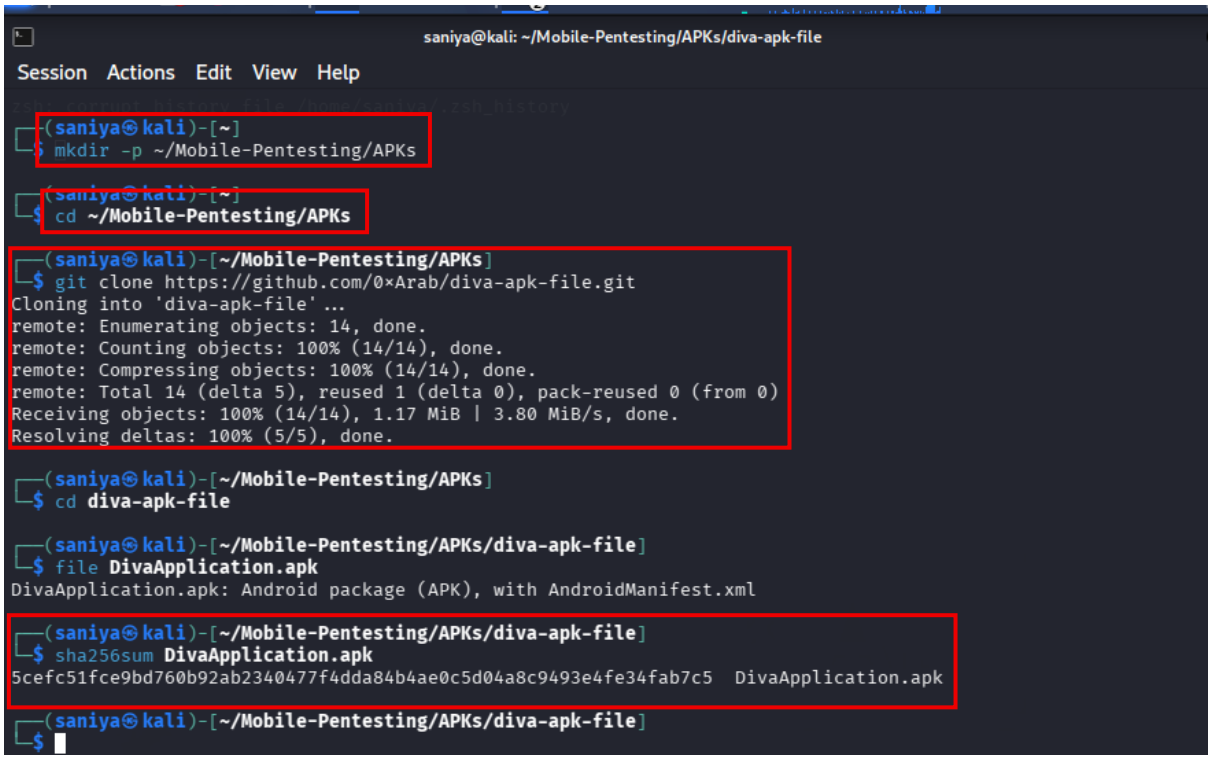
cd diva-apk-file

Step 5: Verify the APK file type

file DivaApplication.apk

Step 6: Calculate the SHA-256 hash

sha256sum DivaApplication.apk



Step 2: APK Reconnaissance

Finding 1: Outdated Android Target SDK Version

Risk Level: Medium

OWASP: M8 – Security Misconfiguration

Objective: Determine what information can be learned about an APK before decompiling it.

Command: aapt dump badging DivaApplication.apk

Description:

During the analysis, the application's target SDK version was found to be 23, which corresponds to Android 6.0 (Marshmallow). The application also has a minimum SDK version of 15. Targeting an older Android version may cause the application to miss newer security protections and use outdated security behavior on modern devices.

Evidence:

sdkVersion:'15'

targetSdkVersion:'23'

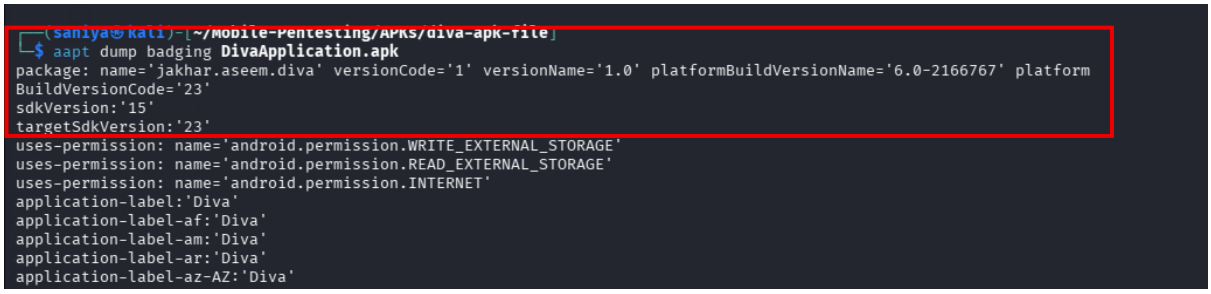


Figure 2: DIVA Application Target SDK Version

Impact

The application may miss newer Android security protections and may use outdated security behavior on modern devices, increasing the risk of security vulnerabilities.

Remediation:

- Update the target SDK version to a recent Android API level.
- Test the application on newer Android versions to ensure compatibility.

- Follow current Android security guidelines and use updated security features.
- Regularly review and update the application's SDK versions to maintain security.

Finding 2: Debuggable Application Enabled

Risk Level: High

OWASP: M8 – Security Misconfiguration

Description:

During the APK reconnaissance, the DIVA application was found to have debugging enabled. This was identified through the application-debuggable entry in the command output. When debugging is enabled, an attacker with suitable access may be able to inspect the application's internal behavior and access sensitive information. This setting can create a security risk if the application is used in a production environment.

Evidence:

application-debuggable

```
application-icon-480:'res/mipmap-xxhdpi-v4/ic_launcher.png'
application-icon-640:'res/mipmap-xxxhdpi-v4/ic_launcher.png'
application: label='Diva' icon='res/mipmap-mdpi-v4/ic_launcher.png'
application-debuggable
launchable-activity: name='jakhar.aseem.diva.MainActivity'  label='Diva' icon=''
feature-group: label=''
  uses-feature: name='android.hardware.faketouch'
  uses-implied-feature: name='android.hardware.faketouch' reason='default feature for all apps'
main
other-activities
supports-screens: 'small' 'normal' 'large' 'xlarge'
supports-any-density: 'true'
```

Figure 1: DIVA Application Debuggable Configuration

Impact

An attacker with suitable access may be able to inspect the application's internal behavior, access sensitive information, or tamper with application data. This can increase the risk of data exposure if debugging is enabled in a production environment.

Remediation

- Disable debugging by setting android:debuggable="false" in AndroidManifest.xml for production builds.
- Use release builds instead of debug builds for production distribution.
- Verify that debugging is disabled before publishing the application.
- Avoid storing sensitive information in the application that could be exposed during debugging.

Finding 3: Broad External Storage Permissions

Risk Level: Medium

M9 – Insecure Data Storage (potential; requires further verification)

Description:

The APK requests both read and write permissions for external storage. These permissions may allow the application to access or modify files on shared storage, depending on the Android version and permission handling. If sensitive information is stored in external storage without proper protection, it may be exposed to other applications or users.

Evidence:

Package name: 'Jakhar.aseem.diva'

uses-permission: name='android.permission.WRITE_EXTERNAL_STORAGE'

uses-permission: name='android.permission.READ_EXTERNAL_STORAGE'

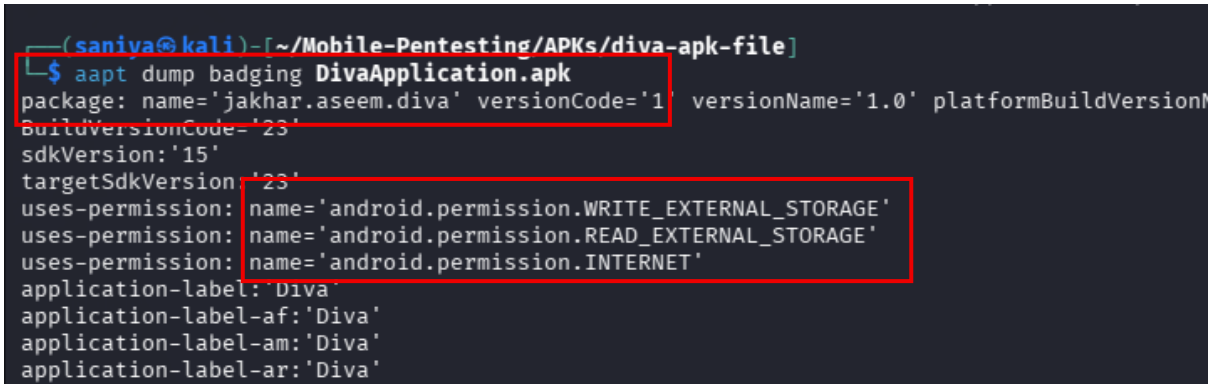


Figure 3: DIVA Application Storage Permissions

Impact

Sensitive information stored in external storage may be exposed to other applications or users with access to shared storage. Unauthorized modification or deletion of files may also occur.

Remediation

- Avoid storing sensitive information in shared external storage.
- Use internal app storage for confidential data.
- Remove unnecessary external storage permissions.
- Apply proper access controls and encryption when storing sensitive data.

- Review the application's storage permissions and use modern Android storage APIs.

Finding 4: Application Information Disclosure

Risk Level: low

Description:

The APK reveals application details such as the package name, application name, version number, and launchable activity. This information can help an analyst understand the application and identify its main entry point before performing further security testing. This is informational and does not independently confirm a vulnerability.

Evidence:

package: name='jakhar.aseem.diva' versionCode='1' versionName='1.0'

application-label:'Diva'

launchable-activity: name='jakhar.aseem.diva.MainActivity'



Figure 4: DIVA Application Package Information

Impact

The exposed application details may help an analyst understand the application's structure, identify its main entry point, and plan further security testing. This information does not directly expose sensitive data or confirm a vulnerability.

Remediation

- Avoid exposing unnecessary application information in public documentation.
- Review application metadata and remove unnecessary details where possible.
- Keep the application's security configuration up to date.

- Treat this information as low risk and focus on protecting sensitive data and application functionality.

Step 3 - Understand APK Structure

Objective: To inspect the internal structure of the DIVA APK and understand the files and resources contained inside it.
Inspect and Extract
unzip -l DivaApplication.apk | head -30
mkdir ~/Mobile-Pentesting/Extracted/DIVA-Raw
unzip DivaApplication.apk -d ~/Mobile-Pentesting/Extracted/DIVA-Raw
cd ~/Mobile-Pentesting/Extracted/DIVA-Raw
ls -lah

Finding 5: APK Internal Structure and Component Identification

Risk Level : low

Owasp: M7 – Insufficient Binary Protections

Objective: To understand the internal structure of the DIVA APK and identify its main components, including application configuration, compiled code, resources, native libraries, and signing information.

Description

In this step, I examined the internal structure of the DIVA APK using the unzip -l command. The APK is a ZIP archive that contains different files and directories required for the application to function.

The following components were identified as important parts of the APK structure:

Component	Purpose
AndroidManifest.xml	Contains application configuration and security settings.
classes.dex	Contains compiled Android application code.
res/	Contains application resources.
assets/	Contains bundled application files, when present.
lib/	Contains native libraries.
META-INF/	Contains signing-related information.
resources.arsc	Contains compiled application resources.

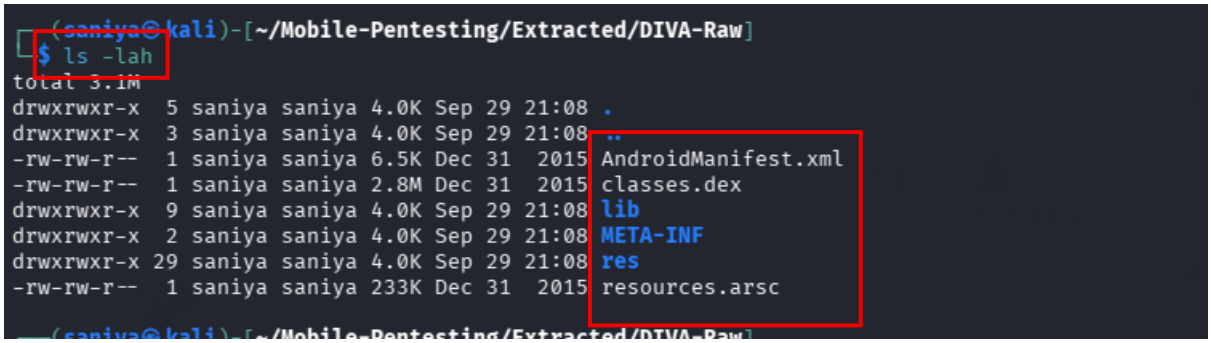


Figure 5: DIVA APK Internal Components

Impact

The APK's internal structure can help an attacker understand the application's components, locate compiled code and resources, and identify areas for further analysis. No direct security impact was confirmed from this finding alone.

Remediation

- Avoid storing sensitive information directly in the APK.
- Use code obfuscation to make reverse engineering more difficult.
- Review the application for exposed secrets and sensitive data.
- Apply appropriate security protections to sensitive functionality.

Finding 6: Hardcoded Sensitive Information in DEX File

Risk Level: High

OWASP: M1 – Improper Credential Usage.

Commands Used:

strings classes.dex | head -30

strings classes.dex | grep -iE 'password|secret|key|http'

Description:

The strings command was used to extract readable text from the classes.dex file. The first command displayed the first 30 readable strings, while the second searched for strings containing words such as password, secret, key, and http.

This process helps identify potentially sensitive information, such as passwords, secret keys, and URLs, stored in the compiled application code.

Command: strings classes.dex | grep -iE 'API Key|API Password|password[[:space:]]*:|secret'

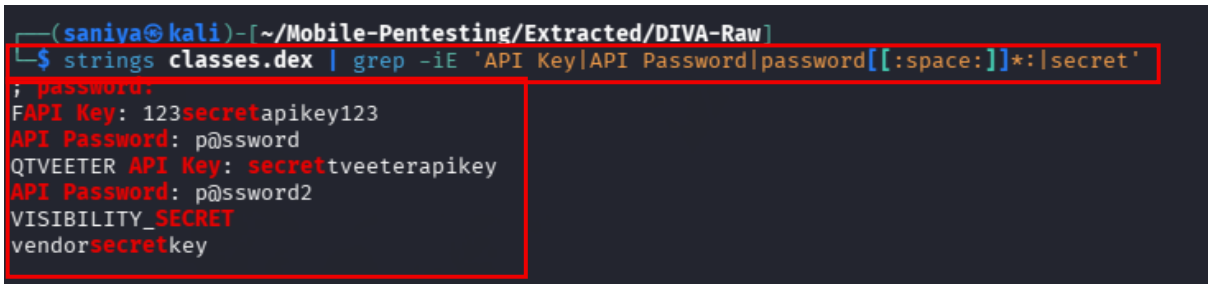


Figure 6: Hardcoded API Keys and Passwords in classes.dex

Impact

Hardcoded sensitive information in the DEX file can be extracted by anyone who decompiles or analyzes the APK. If the exposed API keys or passwords are valid, an attacker may use them to access services, misuse APIs, or gain unauthorized access to protected resources. This could lead to data exposure or unauthorized activity.

Remedy

- Avoid storing passwords, API keys, or secret credentials directly in the application code.
- Store sensitive credentials securely on the server and handle authentication through a backend.
- Revoke or rotate any exposed credentials that are still valid.
- Apply proper authentication and authorization checks to prevent unauthorized access.
- Review the APK for other hardcoded secrets before release.

Step 4: AndroidManifest.xml Analysis

Finding 7: Insecure Application Configuration in AndroidManifest.xml

Risk Level: Medium

OWASP: M9 – Insecure Data Storage (potential).

Commands Used:
cd ~/Mobile-Pentesting/APKs/diva-apk-file

apktool d DivaApplication.apk \
-o ~/Mobile-Pentesting/Extracted/DIVA-Apktool

cd ~/Mobile-Pentesting/Extracted/DIVA-Apktool

cat AndroidManifest.xml

Description

After extracting the APK, I explored the extracted folders and files to understand the application structure. Then, I used APKTool to decode the AndroidManifest.xml file and analyzed it to identify important security settings and permissions. During the analysis, I found that the application had debugging enabled, backup was allowed, and permissions such as READ_EXTERNAL_STORAGE, WRITE_EXTERNAL_STORAGE, and INTERNET were declared. These observations helped me understand the application's configuration and identify potential security risks for further investigation.

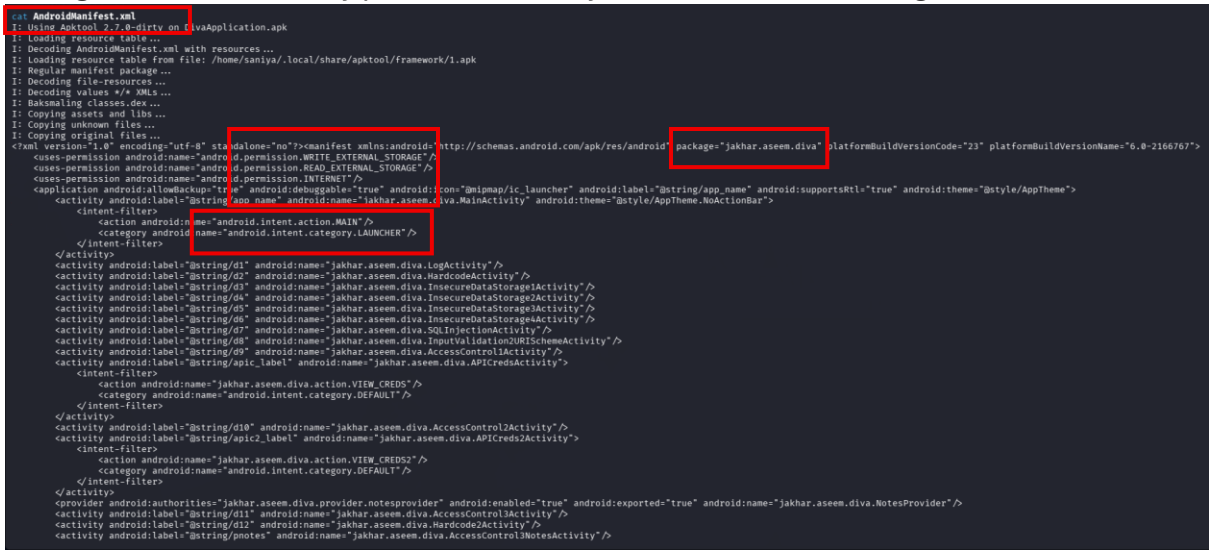


Figure 8: Allow Backup Setting in AndroidManifest.xml

Impact

Debugging enabled may expose the application to additional debugging and inspection risks. Allowing backups may increase the risk of sensitive application data being exposed, depending on the backup configuration and Android version. Broad storage permissions may also increase the risk of unauthorized access to files.

Remedy

- Disable debugging in production builds by setting android:debuggable="false".
- Disable backup if it is not required, or configure backup rules to exclude sensitive data.
- Remove unnecessary storage permissions and use more restricted storage access methods.
- Review application permissions and keep only those required for its functionality.

- Test the application to ensure sensitive data is not exposed through debugging, backups, or storage access.

Step 5 – Analyze res/ and Resources

Finding 8: Sensitive Information Indicators in Application Resources

OWASP Mobile Top 10: M9 – Insecure Data Storage (potential).

Risk Level: Low

Commands Used

cd ~/Mobile-Pentesting/Extracted/DIVA-Apktool

find res -maxdepth 2 -type d

cat res/values/strings.xml

grep -RniE 'password|secret|api.?key|token|http|https' res/

find res -type f -name "*.xml"

Description
After analyzing the AndroidManifest.xml file, I moved to the res/ directory to inspect the application's resources. I explored the folders, reviewed the strings.xml file, and searched for sensitive information such as passwords, API keys, tokens, and URLs. I also listed the XML files to understand the application's resource structure and identify any potentially sensitive information stored in the resources.

Security Principle
Application resources may contain sensitive information, such as API keys, passwords, endpoints, tokens, database details, and configuration data.

Command: grep -RniE 'password|passwd|secret|api.?key|token|http|https|username|credential|database|credit.?card|access.?key|private.?key|client.?secret|endpoint|server' res/

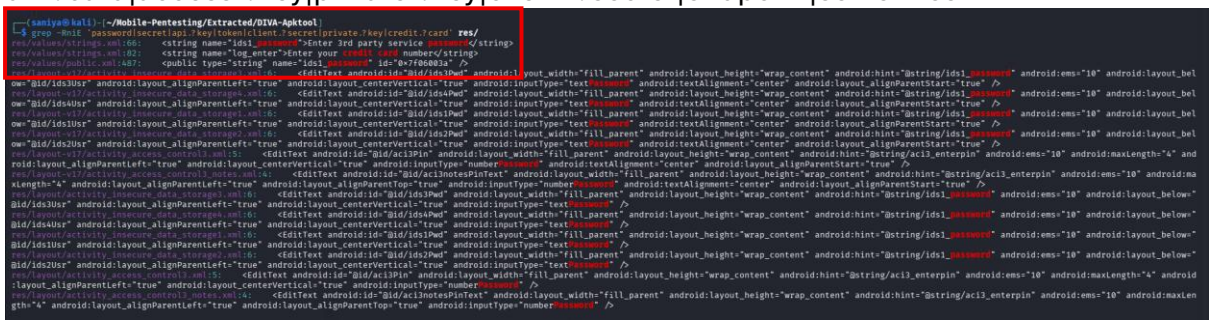


Figure 8: Password and Credit Card References in the res/ Directory

Impact

The presence of sensitive information in application resources may expose details about the application's configuration, endpoints, or credentials. If valid secrets or sensitive data are stored in these resources, an attacker could extract them by analyzing the APK and potentially misuse them.

In this analysis, the search was used to identify potential sensitive information. No confirmed exposed credentials were established from the resource scan alone.

Remedy

- Avoid storing passwords, API keys, tokens, or other secrets directly in application resources.
- Store sensitive credentials securely on the server and retrieve data through authenticated APIs.
- Remove unnecessary sensitive information from resource files.
- Review URLs and configuration details to ensure they do not expose internal systems or confidential information.
- Re-scan the resources before release to check for accidentally exposed secrets.

Step 6 – Analyze Native Libraries (lib/)

Command:

```
cd ~/Mobile-Pentesting/Extracted/DIVA-Raw

find lib -type f

file lib/*/*.so

strings lib/*/*.so | head -50

strings lib/*/*.so | grep -iE 'password|secret|key|token|http|sql|pin'
```

Description

After analyzing the application resources, I moved to the lib/ directory to inspect the native libraries. I used the find command to list the available library files and the file command to identify their types. Then, I used the strings command to extract readable text and searched for potentially sensitive information such as passwords, secret keys, tokens, URLs, SQL statements, and PINs.

Finding 9: Potential Hardcoded Secret in Native Library

OWASP: M1 – Improper Credential Usage (potential).
Risk Level: Medium

Description
During native library analysis, I inspected the .so files inside the lib/ directory using the strings command. I found a readable string cxa atexit in the native library output, along with JNI function names such as Java_jakhar_aseem_diva_DivaJni_access and Java_jakhar_aseem_diva_DivaJni_initiateLaunchSequence.

This indicates that readable values and native code are present in the library. The string olsdfgad;lh may be a hardcoded value, but its purpose needs to be verified before confirming it as a secret.

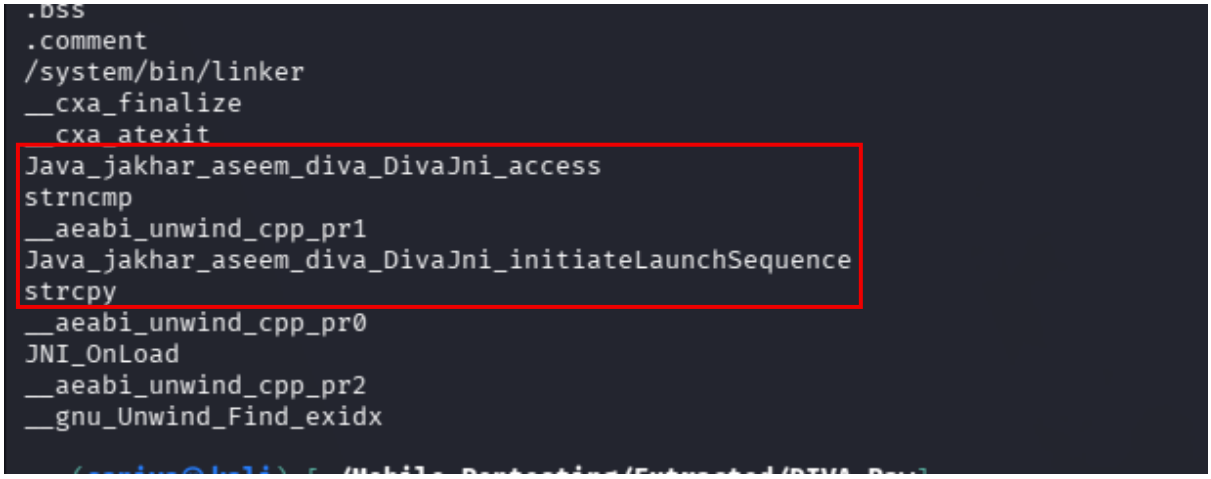


Figure 9: Readable Strings in libdivajni.so

Impact

If the string olsdfgad;lh is a secret or authentication value, an attacker may be able to extract it from the native library and potentially misuse it to access protected functionality or resources.

The presence of JNI functions also reveals native code functionality that may help an attacker analyze the application. However, the purpose of the string has not been confirmed, so no direct security impact has been established.

Remedy

- Investigate the purpose of olsdfgad;lh to determine whether it is a secret or an ordinary string.
- Avoid hardcoding sensitive credentials or secret keys in native libraries.
- If the string is confirmed to be a valid secret, remove it and rotate or revoke it.

- Store sensitive credentials securely on the server and use proper authentication and authorization.
- Review native code for other exposed secrets and sensitive information.

Step 5 – Analyze META-INF/ and APK Signing
Commands:

cd ~/Mobile-Pentesting/Extracted/DIVA-Raw

find META-INF -type f

Description
After inspecting the META-INF/ directory, I used the apksigner command to verify the APK's digital signature and review the certificate information. I checked the signature schemes, signer details, certificate fingerprints, and key size to understand how the application was signed and whether its integrity verification succeeded.

Finding 10: APK Signing Files Identified

OWASP: M7 – Insufficient Binary Protections (potential).

Risk Level: low

Description:

During the APK signing analysis, I inspected the META-INF/ directory and found three files: MANIFEST.MF, CERT.RSA, and CERT.SF. These files contain signing-related information and are used in the APK's signing process.

Evidence to Highlight:

- META-INF/MANIFEST.MF
- META-INF/CERT.RSA
- META-INF/CERT.SF

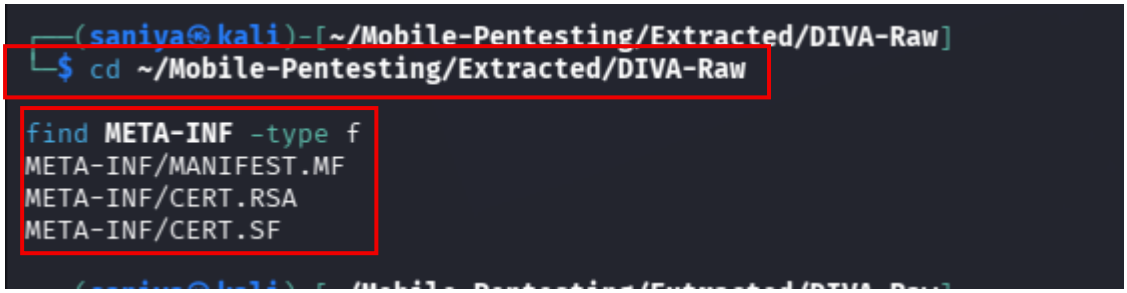


Figure: 10 APK Signing Files Identified

Impact

These files contain information related to the APK's signing process. Their presence alone does not indicate a security vulnerability. However, signing information can help an analyst understand how the APK was signed.

Remedy

- Use a valid and secure signing certificate for production builds.
- Protect the private signing key from unauthorized access.
- Verify the APK's signature before distributing the application.
- Review the signing configuration to ensure it follows security best practices.

Finding 11 : APK Signed with a Debug Certificate

Severity: Medium

OWASP : M7 – Insufficient Binary Protections (potential).

Command: apksigner verify --verbose --print-certs ~/Mobile-Pentesting/APKs/diva-apk-file/DivaApplication.apk

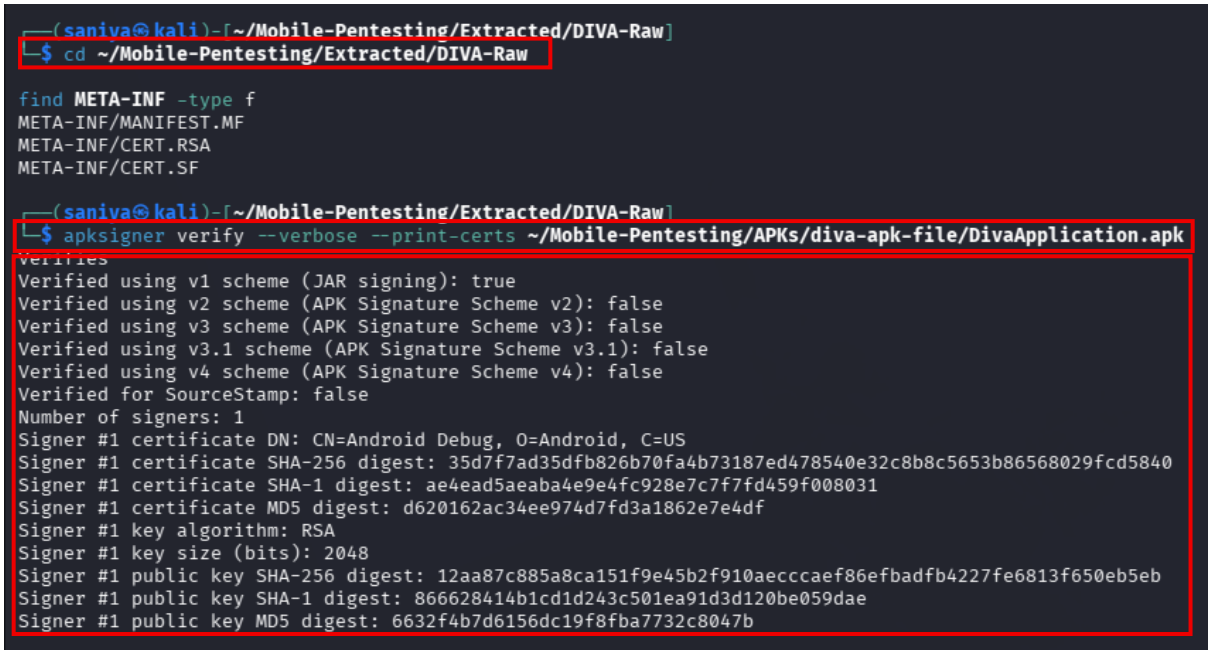


Figure 11: APK Signature Verification and Certificate Information

What I Found

- Signature verification: Successful (Verifies)

- Signer DN: CN=Android Debug, O=Android, C=US
- Key algorithm: RSA
- Key size: 2048 bits
- V1 signing: Yes
- V2 signing: No
- V3 signing: No
- V3.1 signing: No
- V4 signing: No
- Number of signers: 1
- Certificate fingerprints: SHA-256, SHA-1, and MD5 were displayed.

Impact

Using a debug certificate for signing may indicate that the APK was built for testing rather than production. If used in a production environment, it may reduce trust in the application's signing process and create security risks if the signing key is not properly protected.

Remedy

- Use a secure, production signing certificate for release builds.
- Keep the private signing key protected from unauthorized access.
- Avoid using debug certificates for production applications.
- Verify the APK's signature before distributing it.

Step 6 – JADX Decompilation

Objective

JADX is used to convert the APK's DEX bytecode into readable Java-like source code. This helps analyze the application's code and identify potential security issues.

1. Check the APK file:

ls -l ~/Mobile-Pentesting/APKs/diva-apk-file/

2. Decompile the APK using JADX:

jadx -d ~/Mobile-Pentesting/Extracted/DIVA-JADX ~/Mobile-Pentesting/APKs/diva-apk-file/DivaApplication.apk

3. Navigate to the decompiled folder:

cd ~/Mobile-Pentesting/Extracted/DIVA-JADX

4. List the decompiled Java source files:

ls sources/jakhar/aseem/diva/

Finding 12: Successful APK Decompilation and Source Code Identification

OWASP Mobile Top 10: No direct category assigned.

Risk Level: low

Description:

After successfully decompiling the DIVA APK using JADX, I explored the generated source code directory and listed the Java files. I found several application activities, including HardcodeActivity.java, APICredsActivity.java, InsecureDataStorage1Activity.java, InsecureDataStorage2Activity.java, and SQLInjectionActivity.java. These files helped me identify different parts of the application for further security analysis.

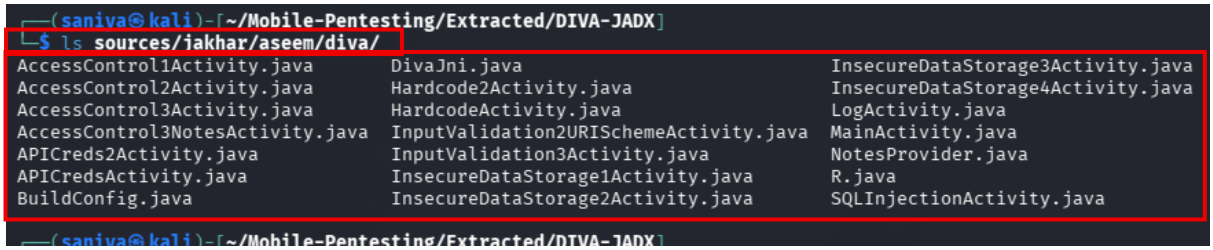


Figure 12: Decompiled Java Source Files in JADX

Impact

Successful APK decompilation allows analysts to view the application's code and understand its functionality. This may help attackers identify potential weaknesses, but decompilation alone does not confirm a security vulnerability.

Remedy

- Use code obfuscation to make reverse engineering more difficult.
- Avoid storing sensitive information directly in the application code.
- Review the application code for security weaknesses before release.
- Protect sensitive functionality with proper server-side authentication and authorization.

Step 7: Manual Source-Code Security Analysis

Description

After successfully decompiling the APK using JADX, I reviewed the Java source files to identify potential security vulnerabilities. I focused on how the application stores sensitive data, handles logs, and manages API credentials. During the analysis, I identified issues related to plaintext credential storage, sensitive information in logs, and hardcoded API credentials.

Finding 13: Insecure SQLite Database Storage

OWASP Mobile Top 10: M9 – Insecure Data Storage.

Risk Level : High

Source File: InsecureDataStorage2Activity.java

Command:

cat sources/jakhar/aseem/diva/InsecureDataStorage2Activity.java

Vulnerable Code:

INSERT INTO myuser VALUES ('username', 'password');

Description:

During source-code analysis, I reviewed InsecureDataStorage2Activity.java and found that the application stores the username and password in an SQLite database without encryption. This may expose the credentials if an attacker gains access to the application's database.

```
┌──(saniya㉿kali)-[~/Mobile-Pentesting/Extracted/DIVA-JADX]
└─$ cat sources/jakhar/aseem/diva/InsecureDataStorage2Activity.java
package jakhar.aseem.diva;

import android.database.sqlite.SQLiteDatabase;
import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;

/* JADX INFO: loaded from: classes.dex */
public class InsecureDataStorage2Activity extends AppCompatActivity {
    private SQLiteDatabase mDB;

    @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity, android.support
.v4.app.BaseFragmentActivityDonut, android.app.Activity
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        try {
            this.mDB = openOrCreateDatabase("ids2", 0, null);
            this.mDB.execSQL("CREATE TABLE IF NOT EXISTS myuser(user VARCHAR, password VARCHAR);");
        } catch (Exception e) {
            Log.d("Diva", "Error occurred while creating database: " + e.getMessage());
        }
        setContentView(R.layout.activity_insecure_data_storage2);
    }

    public void saveCredentials(View view) {
        EditText usr = (EditText) findViewById(R.id.ids2Usr);
        EditText pwd = (EditText) findViewById(R.id.ids2Pwd);
        try {
            this.mDB.execSQL("INSERT INTO myuser VALUES ('" + usr.getText().toString() + "', '" + pwd.getText().toS
tring() + "');");
            this.mDB.close();
        } catch (Exception e) {
            Log.d("Diva", "Error occurred while inserting into database: " + e.getMessage());
        }
        Toast.makeText(this, "3rd party credentials saved successfully!", 0).show();
    }
}
```

Figure 13: Plaintext Credentials Stored in SQLite Database

Impact

Storing usernames and passwords in an SQLite database without encryption may expose sensitive credentials if an attacker gains access to the database. This could lead to unauthorized access to user accounts and possible data exposure.

Remedy

- Avoid storing passwords in plain text.
- Use secure password hashing with a unique salt when storing passwords.
- Encrypt sensitive data stored in the database when necessary.
- Restrict access to the database and protect application data.
- Avoid storing sensitive information unless it is required.

Finding 14: Insecure Temporary File Storage

OWASP: M9 – Insecure Data Storage.

Risk Level: High

Source File: InsecureDataStorage3Activity.java

Command Used:

cat sources/jakhar/aseem/diva/InsecureDataStorage3Activity.java

Description:

During manual source-code analysis using JADX, I reviewed InsecureDataStorageActivity.java and found that the application stores the username and password in a temporary file inside the application's data directory. The file is marked as readable and writable, and the credentials are written in plaintext. This may expose sensitive information if an attacker gains access to the application's files.

Vulnerable Code:

File info = File.createTempFile("uinfo", "tmp", dir);

info.setReadable(true);

info.setWritable(true);

FileWriter fw = new FileWriter(info);

fw.write(usr.getText().toString() + ":" + pwd.getText().toString());

```
┌──(saniya㉿kali)-[~/Mobile-Pentesting/Extracted/DIVA-JADX]
└─$ cat sources/jakhar/aseem/diva/InsecureDataStorage3Activity.java
package jakhar.aseem.diva;

import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;
import java.io.File;
import java.io.FileWriter;

/* JADX INFO: loaded from: classes.dex */
public class InsecureDataStorage3Activity extends AppCompatActivity {
    @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity, android.support.v4.app.BaseFragmentActivityDonut, android.app.Activity
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_insecure_data_storage3);
    }

    public void saveCredentials(View view) {
        EditText usr = (EditText) findViewById(R.id.ids3Usr);
        EditText pwd = (EditText) findViewById(R.id.ids3Pwd);
        File ddir = new File(getApplicationInfo().dataDir);
        try {
            File uinfo = File.createTempFile("uinfo", "tmp", ddir);
            uinfo.setReadable(true);
            uinfo.setWritable(true);
            FileWriter fw = new FileWriter(uinfo);
            fw.write(usr.getText().toString() + ":" + pwd.getText().toString() + "\n");
            fw.close();
            Toast.makeText(this, "3rd party credentials saved successfully!", 0).show();
        } catch (Exception e) {
            Toast.makeText(this, "File error occurred", 0).show();
            Log.d("Diva", "File error: " + e.getMessage());
        }
    }
}

┌──(saniya㉿kali)-[~/Mobile-Pentesting/Extracted/DIVA-JADX]
```

Figure 14: Plaintext Credentials Stored in a Temporary File

Impact

Storing usernames and passwords in a temporary file in plain text may expose sensitive information if an attacker gains access to the application's files. Making the file

readable and writable may increase the risk of unauthorized access to the stored credentials.

Remedy

- Avoid storing usernames and passwords in temporary files.
- Do not store sensitive information in plain text.
- Use secure storage methods and restrict file access.
- Remove temporary files containing sensitive data when they are no longer needed.
- Avoid making sensitive files readable or writable unnecessarily.

Finding 15: Sensitive Information in Application Logs

OWASP: M9 – Insecure Data Storage.

Risk Level: High

Source File: LogActivity.java

Command Used:

cat sources/jakhar/aseem/diva/LogActivity.java

Sensitive Data: Credit card number entered by the user.

Description:

During manual source-code analysis, I reviewed LogActivity.java and found that the application writes the credit card number entered by the user into the Android system log when an exception occurs. The Log.e() method records the entered value, which may expose sensitive payment information to anyone who gains access to the relevant logs.

Vulnerable Code:

```
Log.e("diva-log", "Error while processing transaction with credit card: " +
cctxt.getText().toString());
```



```
└─$ cat sources/jakhar/aseem/diva/LogActivity.java
package jakhar.aseem.diva;

import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.util.Log;
import android.view.View;
import android.widget.EditText;
import android.widget.Toast;

/* JADX INFO: loaded from: classes.dex */
public class LogActivity extends AppCompatActivity {
    @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity, android.support.v4.app.BaseFragmentActivityDonut, android.app.Activity
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_log);
    }

    public void checkout(View view) {
        EditText cctxt = (EditText) findViewById(R.id.ccText);
        try {
            processCC(cctxt.getText().toString());
        } catch (RuntimeException e) {
            Log.e("diva-log", "Error while processing transaction with credit card: " + cctxt.getText().toString());
            Toast.makeText(this, "An error occured. Please try again later", 0).show();
        }
    }

    private void processCC(String ccstr) {
        RuntimeException e = new RuntimeException();
        throw e;
    }
}
```

Figure 15: Credit Card Information Written to Android System Logs

Impact

Logging credit card numbers may expose sensitive payment information to anyone who gains access to the application's logs. This could lead to misuse of payment details and potential financial harm to the user.

Remedy

- Never log credit card numbers or other sensitive information.
- Remove sensitive data from error messages and application logs.
- Log only general error details needed for debugging.
- Use secure payment processing methods to protect cardholder information.
- Review the application logs to ensure sensitive data is not exposed.

Finding 16: Hardcoded API Credentials in Application Code

OWASP: M1 – Improper Credential Usage.

Risk Level: Critical

Source File: APICredsActivity.java

Command Used:

cat sources/jakhar/aseem/diva/APICredsActivity.java

Description

During manual source-code analysis, I reviewed APICredsActivity.java and found that the application contains an API key, username, and password directly in the source

code. These credentials are displayed in the application using the setText() method. Anyone who obtains the APK can extract these values through static analysis.

Vulnerable Code:

apicview.setText("API Key: 123secretapikey123\nAPI User name: diva\nAPI Password: p@ssword");

```
┌──(saniya㉿kali)-[~/Mobile-Pentesting/Extracted/DIVA-JAVA]
└─$ cat sources/jakhar/aseem/diva/APICredsActivity.java
package jakhar.aseem.diva;

import android.os.Bundle;
import android.support.v7.app.AppCompatActivity;
import android.widget.TextView;

/* JADX INFO: loaded from: classes.dex */
public class APICredsActivity extends AppCompatActivity {
    @Override // android.support.v7.app.AppCompatActivity, android.support.v4.app.FragmentActivity, android.support.v4.app.BaseFragmentActivityDonut, android.app.Activity
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_apicreds);
        TextView apicview = (TextView) findViewById(R.id.apicredstxtview);
        apicview.setText("API Key: 123secretapikey123\nAPI User name: diva\nAPI Password: p@ssword");
    }
}
```

Figure 16: API Credentials Embedded in Application Source Code

Impact

Hardcoded API credentials can be extracted by anyone who obtains the APK. If the credentials are valid, an attacker may use them to access API services without authorization, potentially exposing sensitive data or misusing the service.

Remedy

- Avoid storing API keys, usernames, and passwords directly in application code.
- Store sensitive credentials securely on the server.
- Revoke or change any exposed credentials that are still valid.
- Use proper authentication and authorization to protect API access.
- Review the application for other hardcoded secrets before release.

Step 8 – APKTool + Smali Analysis

Objective

Smali is a human-readable representation of Dalvik bytecode. In this step, I used APKTool to inspect the Smali code and search for hardcoded sensitive information inside the application.

Finding 17: Hardcoded Secret Key in Smali Code

OWASP– Improper Credential Usage.

Risk Level: High

Source File: HardcodeActivity.smali

Command Used:

cat ~/Mobile-Pentesting/Extracted/DIVA-Apktool/smali/jakhar/aseem/diva/HardcodeActivity.smali

Description

During manual Smali analysis, I reviewed HardcodeActivity.smali and found that the application contains a hardcoded secret key, vendorsecretkey. The application compares the user-entered value with this key. If the entered value matches, the application displays an access-granted message. Since the key is stored directly in the application's bytecode, it can be extracted through static analysis.

Vulnerable Code:

const-string v2, "vendorsecretkey"

invoke-virtual {v1, v2}, Ljava/lang/String;->equals(Ljava/lang/Object;)Z

```
.line 19
.local v0, "hckey":Landroid/widget/EditText;
invoke-virtual {v0}, Landroid/widget/EditText;→getText()Landroid/text/Editable;

move-result-object v1

invoke-virtual {v1}, Ljava/lang/Object;→toString()Ljava/lang/String;

move-result-object v1

const-string v2, "vendorsecretkey"

invoke-virtual {v1, v2}, Ljava/lang/String;→equals(Ljava/lang/Object;)Z

move-result v1

if-eqz v1, :cond_0
```

Figure 17: Hardcoded Secret Key in HardcodeActivity.smali

Impact

The hardcoded secret key can be extracted from the application's Smali code. An attacker may use the key to bypass the application's access check and gain access to functionality protected by it.

Remedy

- Avoid storing secret keys directly in the application's code.
- Use server-side authentication to verify access.
- Remove the hardcoded key and use a secure authentication method.
- If the key is used elsewhere, replace it and review for unauthorized access.

- Test the application to ensure access cannot be granted without proper authorization.

Conclusion

The security analysis of the DIVA Android application was conducted using static analysis tools such as APKTool, JADX, and command-line utilities. The assessment identified several security weaknesses, including hardcoded API credentials, insecure SQLite database storage, insecure temporary file storage, sensitive information in application logs, and a hardcoded secret key.

The findings show that storing sensitive information in plain text, embedding credentials in application code, and using insecure application configurations may expose the application to unauthorized access and data leakage.

Based on the analysis, recommendations were provided to improve data protection, secure credential handling, strengthen application configuration, and reduce potential security risks.

Overall, this assessment helped identify areas for improvement and provided a foundation for enhancing the security of the DIVA application.